Practical End-to-End RAG Implementation

Customer Support Assistant Using a Knowledge Base

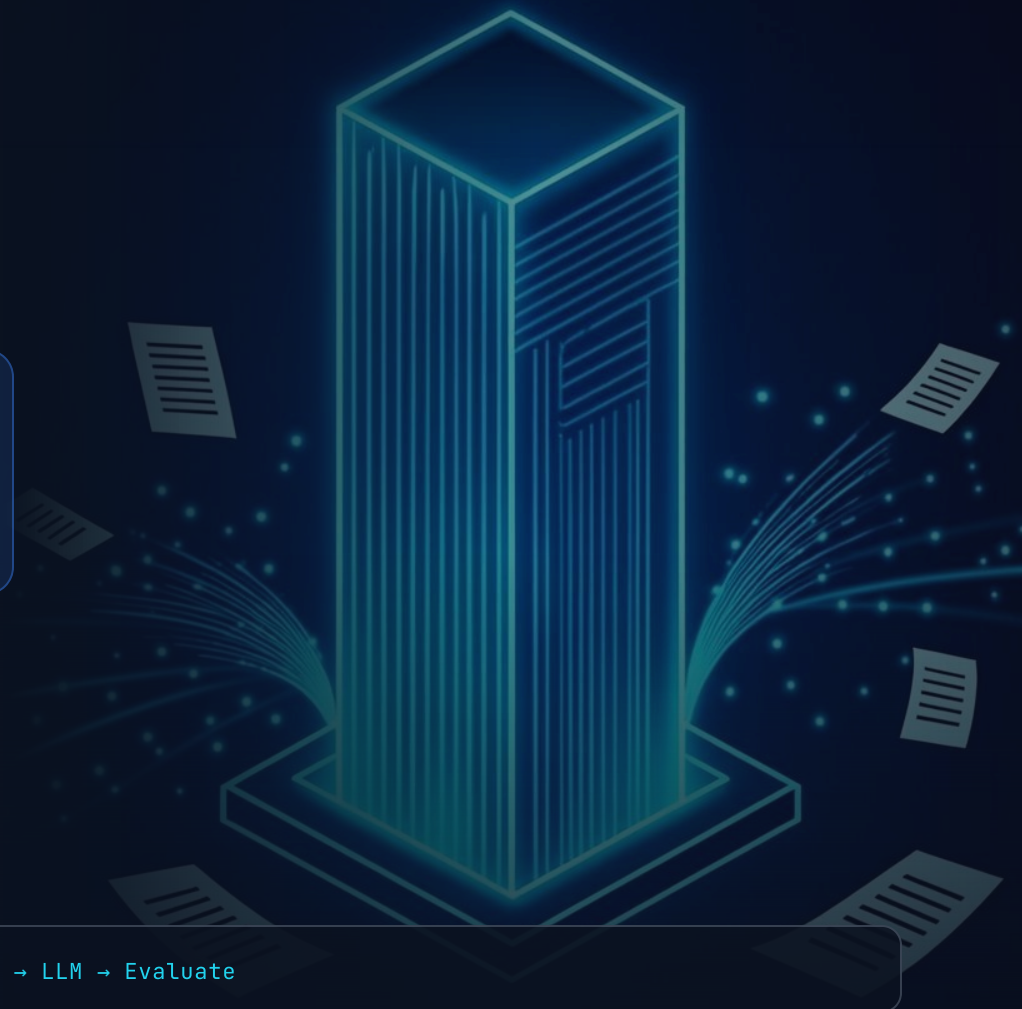
Module 2 — Lecture 5

MODULE 2 · LECTURE 5 | DURATION · 60 MIN | SYLLABUS · 2.3 PRACTICAL SESSION

OPENING QUESTION

"In the previous four lectures, every RAG architecture contained boxes called **Chunking, Embedding, Vector Database, Retriever** and **LLM**."

Today we run it. We replace every one of those boxes with working Python code.



Documents → Chunk → Embed → FAISS → Retrieve → Augment → LLM → Evaluate

PRACTICAL SCENARIO CUSTOMER SUPPORT RAG ASSISTANT

What Are We Building?

THE USER ASKS

"My router shows a red Internet light. What should I do?"

OUR KNOWLEDGE BASE CONTAINS

"If the Internet LED is red, verify the WAN cable, restart the router, wait for synchronization, and contact support if the condition persists."

THE RAG APPLICATION SHOULD

1. Question
2. Find the correct troubleshooting information
3. Give it to the LLM
4. Generate a grounded answer

LEARNING OUTCOMES — BY THE END YOU CAN

Stack & knowledge
Install the basic RAG software stack · create and load a knowledge base

Indexing
Split documents into chunks · embed every chunk · read the embedding matrix shape · create a FAISS vector index

Retrieval
Embed the user question · semantic similarity search · retrieve Top-k chunks · inspect evidence before generation

Generation
Construct an augmented prompt · generate an answer with a small open LLM

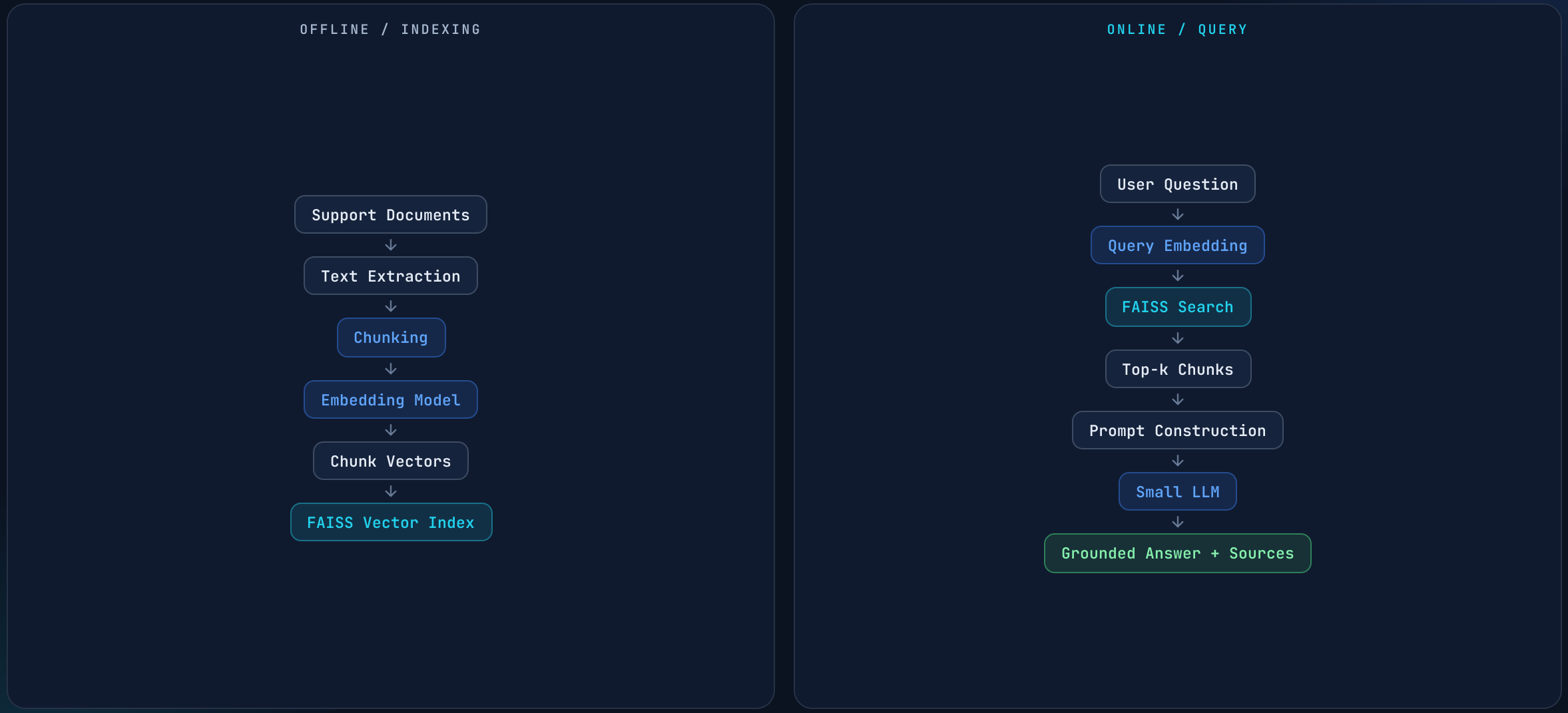
Trust
Compare standalone LLM vs RAG response · add source attribution · implement an abstention rule

Evaluation
Evaluate retrieval and generation separately · diagnose common failures · map the code to Lectures 1–4

One hour · one notebook · **one working RAG assistant** — every concept from Lectures 1–4 becomes executable.

BLUEPRINT TWO PHASES · ONE INDEX

Complete Architecture

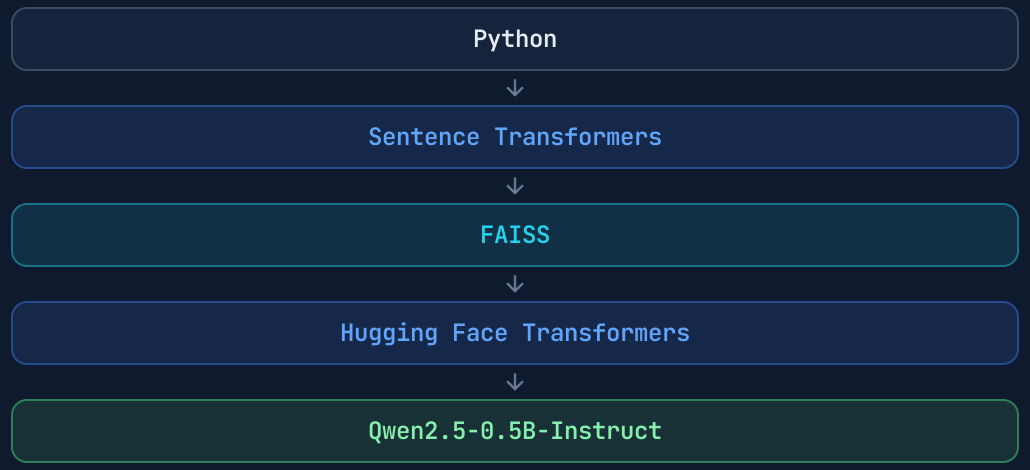


Tell students: "Today we will execute every arrow in this diagram."

TOOLING SIMPLE GOOGLE COLAB IMPLEMENTATION

Technology Stack

FIVE LAYERS, BOTTOM TO TOP



REQUIREMENT	TOOL
Programming	Python
Embeddings	Sentence Transformers
Vector search	FAISS
Generator LLM	Qwen2.5-0.5B-Instruct
Environment	Google Colab

WHY NO LANGCHAIN INITIALLY?

Because students should **understand RAG** before learning a framework that **automates RAG**.

SETUP · STEP 1 ENVIRONMENT + GPU CHECK

Google Colab Setup

CREATE

Google Colab → **New Notebook**

IF GPU IS AVAILABLE

Runtime → Change runtime type → **T4 GPU**

THEN VERIFY — COLAB CELL

```
import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

EXPECTED — CONCEPTUALLY

CUDA available: **True**

GPU: **Tesla T4**

No GPU? Still fine.

The **embedding / retrieval portion** also works without a GPU — only generation benefits from the T4.

SETUP · STEP 2 COLAB CELL 1

Install Required Libraries

INSTALL

```
!pip -q install sentence-transformers \
    faiss-cpu transformers accelerate
```

THEN IMPORT

```
import numpy as np
import faiss
import torch

from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForCausalLM
```

- `sentence-transformers` → convert text into embeddings
- `faiss` → similarity search
- `transformers` → load generator LLM
- `torch` → model execution

Four imports, four jobs — **embeddings, search, generation, execution**. Every later cell uses only these.

COLAB CELL 2 SIX SMALL SUPPORT DOCUMENTS

The Knowledge Base

"The same code can later be connected to a publicly available product manual or FAQ. We keep the first document set small so that every retrieval result can be manually inspected."

DOCUMENTS — PART 1 OF 2

```
documents = [
  {"source": "Router Setup Guide",
   "text": "To install the router, connect the WAN
     cable from the modem to the blue WAN port.
     Connect the power adapter and wait
     approximately two minutes for the router
     to start."},

  {"source": "Internet LED Troubleshooting",
   "text": "If the Internet LED is red, first verify
     that the WAN cable is firmly connected.
     Restart both the modem and router. Wait
     for the Internet LED to stabilize. If it
     remains red, contact your Internet service
     provider or technical support."},

  {"source": "Wi-Fi Password Guide",
   "text": "The default Wi-Fi network name and
     password are printed on the router label.
     The Wi-Fi password can be changed from
     the wireless settings page after logging
     into the administration portal."},
```

DOCUMENTS — PART 2 OF 2

```
  {"source": "Factory Reset Guide",
   "text": "To reset the router to factory settings,
     keep the router powered on and press the
     reset button for approximately ten
     seconds. The router will restart and all
     customized settings will be erased."},

  {"source": "Firmware Update Guide",
   "text": "Firmware updates can be installed from
     the administration portal. Back up the
     router configuration before updating
     firmware and do not disconnect power
     during the update."},

  {"source": "Warranty Policy",
   "text": "The router includes a limited hardware
     warranty. Physical damage, unauthorized
     modification and damage caused by
     improper power supplies may not be
     covered."}
]
```

SYLLABUS BRIDGE COLAB CELL 3

Classroom KB vs Public KB — and Inspect First

FOR THE SYLLABUS, REPLACE THE CLASSROOM KB WITH

- Public Product Manual
- Public FAQ
- Open Technical Documentation
- Manufacturer Support Guide

THE RAG CODE REMAINS

Load → Extract → Chunk → Embed → Store → Retrieve

Only **the knowledge source** changes.

IMPORTANT LESSON

"RAG architecture is independent of whether knowledge comes from **six paragraphs** or **six thousand documents**."

INSPECT THE KNOWLEDGE BASE

```
print("Number of documents:", len(documents))

for i, doc in enumerate(documents):
    print("\nDocument", i)
    print("Source:", doc["source"])
    print(doc["text"].strip())
```

Ask: *"Why should we inspect data before embedding?"*

Because bad data will produce bad retrieval.

Bad Data → Bad Retrieval

RECALL LECTURE 3

INDEXING · CHUNKING A CHUNKING FUNCTION, ON PURPOSE

Why Chunk Again?

Our six documents are **already short**. But real documents may be:

10	50	500
PAGES	PAGES	PAGES

Therefore we need a **chunking function** — the same one we would run on a 500-page manual.

```
chunk_size = 60 words
overlap    = 10 words
```

FOR DEMONSTRATION

```python
def chunk_text(text, chunk_size=60, overlap=10):
    words = text.split()

    chunks = []
    start = 0

    while start < len(words):
        end = start + chunk_size
        chunk = " ".join(words[start:end])
        chunks.append(chunk)

        start += chunk_size - overlap

    return chunks
```

Explain carefully: 60-word windows stepping forward 50 words each time — `intentionally simple for teaching`.

INDEXING · COLAB CELL FROM DOCUMENTS TO (TEXT + SOURCE) PAIRS

Apply Chunking

RUN + INSPECT

```
chunks = []

for doc in documents:
    doc_chunks = chunk_text(
        doc["text"],
        chunk_size=60,
        overlap=10
    )

    for chunk in doc_chunks:
        chunks.append({
            "text": chunk,
            "source": doc["source"]
        })

print("Total chunks:", len(chunks))

for i, chunk in enumerate(chunks):
    print(f"\nChunk {i}")
    print("Source:", chunk["source"])
    print(chunk["text"])
```

CONCEPTUALLY

Document 1

↓ Chunk 1 · Chunk 2

Document 2

↓ Chunk 3 · Chunk 4

EACH CHUNK NOW HAS

TEXT + SOURCE

Text: "If the Internet LED is red…"
Source: "Internet LED Troubleshooting"

Later we add: **EMBEDDING**

INDEXING · EMBEDDINGS ALL-MINILM-L6-V2 · 384 DIMENSIONS

Load the Model, Embed Every Chunk

SMALL, POPULAR RETRIEVAL MODEL

```
embedding_model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
```

Text → all-MiniLM-L6-v2 → Numerical Vector

This model produces **384-dimensional** embeddings.

GENERATE CHUNK EMBEDDINGS

```
chunk_texts = [chunk["text"] for chunk in chunks]

chunk_embeddings = embedding_model.encode(
    chunk_texts,
    convert_to_numpy=True,
    normalize_embeddings=True
)

print("Embedding matrix shape:",
      chunk_embeddings.shape)

(6, 384)
```

"What does 6 mean?" Six chunks.

"What does 384 mean?" 384 numerical features per embedding vector.

EMBEDDINGS · INTUITION MEANING BECOMES GEOMETRY

A Visual Mental Model — and Why Normalize

```
Chunk 1 · "Install router…"
↓
[0.021, -0.154, …, 0.072]
```

```
Chunk 2 · "Internet LED red…"
↓
[0.181, -0.271, …, 0.109]
```

```
Chunk 3 · "Wi-Fi password…"
↓
[…]
```

MEMORY SENTENCE

"Embeddings turn text meaning into **searchable vectors**."

Now the computer can compare chunks **numerically**.

WHY normalize_embeddings=True?

This scales vectors approximately to **unit length**. With normalized vectors:

$$\text{Dot Product} \approx \text{Cosine Similarity}$$

This makes FAISS **inner-product** search convenient.

RECALL LECTURE 3 — DO NOT DERIVE AGAIN

$$\cos(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

If vectors are already unit-length, the denominator is 1 — so inner product **is** cosine similarity.

INDEXING · FAISS VECTORS IN, IDS OUT

Create the FAISS Index — and Know What It Stores

COLAB CELL — BUILD THE INDEX

```
# find embedding dimension
dimension = chunk_embeddings.shape[1]
print("Dimension:", dimension)

# create index
index = faiss.IndexFlatIP(dimension)

# add vectors
index.add(chunk_embeddings.astype("float32"))

# check
print("Vectors stored:", index.ntotal)
```

FAISS INDEX — CONCEPTUALLY

```
Vector 0 → Chunk 0
Vector 1 → Chunk 1
Vector 2 → Chunk 2
...
```

Important: FAISS stores/searches **vectors only**. Our Python list retains chunk **text** and **source**.

"Is FAISS a vector database?" More precisely, FAISS is a **library / index** for efficient vector similarity search — full vector-database systems add persistence, metadata operations, networking and scaling.

`FAISS result ID → chunks[result_id] → Text + Source`

This is a **very important concept** — the ID is our bridge back to readable evidence.

RETRIEVAL · ONLINE YOUR FIRST ACTUAL RETRIEVER

Create the Retrieval Function

COLAB CELL — RETRIEVER

```python
def retrieve(query, k=3):

    query_embedding = embedding_model.encode(
        [query],
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")

    scores, indices = index.search(
        query_embedding,
        k
    )

    results = []

    for score, idx in zip(scores[0], indices[0]):
        results.append({
            "score": float(score),
            "text": chunks[idx]["text"],
            "source": chunks[idx]["source"]
        })

    return results
```

1 · **EMBED** Encode the question with the **same model** and normalization as the chunks.

2 · **SEARCH** index.search returns **scores and indices** for the Top-k nearest vectors.

3 · **MAP IDs** zip scores with indices, then resolve **chunks[idx]** back to text + source.

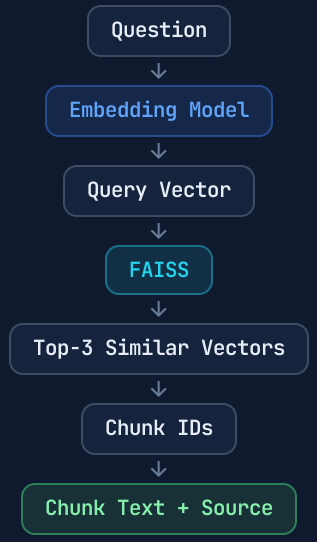
4 · **RETURN** A list of **score / text / source** dicts — evidence, ready to inspect.

This is your **first actual retriever** — twelve lines implementing all of Lecture 2's retrieval story.

RETRIEVAL · TRACE + TEST BEFORE ADDING AN LLM — EXTREMELY IMPORTANT

Trace It, Then Test Retrieval First

QUERY: "My router Internet light is red."



DIRECTLY IMPLEMENTS LECTURES 2 AND 3

TEST — NO LLM YET

```
query = "My router Internet light is red.
         What should I do?"

results = retrieve(query, k=3)

for rank, result in enumerate(results, start=1):
    print(f"\nRank {rank}")
    print("Score:", round(result["score"], 4))
    print("Source:", result["source"])
    print("Text:", result["text"])

# expected highest result:
#   "Internet LED Troubleshooting"
```

Ask before using the LLM: *"Did our retriever actually find the correct information?"* Do **not** immediately move to generation.

MEMORY ANCHOR

"Inspect retrieval before blaming generation."

If Top-1 is wrong, the first debugging target is **retrieval — not the LLM**.

RETRIEVAL · STRESS TEST

ONE QUERY IS ANECDOTE — FOUR IS A PATTERN

Try Multiple Queries

COLAB CELL — FOUR QUESTIONS

```
test_queries = [
    "How do I reset my router?",
    "Where can I find the Wi-Fi password?",
    "Can I disconnect power during firmware update?",
    "What damage is not covered under warranty?"
]

for q in test_queries:
    print("\nQUESTION:", q)

    results = retrieve(q, k=2)

    for r in results:
        print(round(r["score"], 3),
              r["source"])
```

WHAT STUDENTS SEE — TOP SOURCES PER QUESTION

"How do I reset my router?"	→ Factory Reset Guide
"Where can I find the Wi-Fi password?"	→ Wi-Fi Password Guide
"Can I disconnect power during firmware update?"	→ Firmware Update Guide
"What damage is not covered under warranty?"	→ Warranty Policy

EACH ROW PRINTS TWO RANKED SOURCES WITH SCORES — EXACT SCORES VARY

Students can now see **semantic retrieval working** — different wording, correct document, every time.

GENERATION TWO MODELS · TWO JOBS

Load the Generator LLM

CONSISTENT WITH OUR EARLIER LABORATORY

```
model_name = "Qwen/Qwen2.5-0.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(
    model_name
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
```

"The **embedding model retrieves**. **Qwen generates**." — two different models serving different jobs.

STUDENTS MUST NOT CONFUSE:

Embedding Model

Text → Vector

PURPOSE: RETRIEVAL

Generator LLM

Question + Evidence → Natural-language answer

PURPOSE: GENERATION

MEMORY STATEMENT

”Retriever finds.
Generator writes.”

AUGMENTATION EVIDENCE + INSTRUCTIONS + QUESTION

Construct the Context, Then the Grounded Prompt

CONTEXT

```python
def build_context(results):

    context_parts = []

    for i, result in enumerate(results, start=1):
        context_parts.append(
            f"[Source {i}: {result['source']}]\n"
            f"{result['text']}"
        )

    return "\n\n".join(context_parts)


context = build_context(results)
print(context)
```

PROMPT

```python
def build_prompt(question, context):

    return f"""
You are a customer-support assistant.

Answer the question using only the
supplied context.

If the context does not contain enough
information, say: "I do not have enough
information in the provided knowledge base."

Cite the source name used in your answer.

CONTEXT:
{context}

QUESTION:
{question}
"""
```

OUTPUT BEGINS:

```
[Source 1: Internet LED Troubleshooting]
If the Internet LED is red, first verify…
```

using only supplied context
Encourages **grounding**.

abstention instruction
Allows the model **not to invent**.

citation instruction
Improves **traceability**.

MENTAL MODEL EVIDENCE IS NOT ENOUGH BY ITSELF

Why Prompt Design Still Matters in RAG

RAG gives the model

EVIDENCE — retrieved chunks with sources, placed inside the prompt.

Prompt engineering tells it

HOW TO USE THE EVIDENCE — stay grounded, abstain when unsure, cite the source.

RAG = Retrieval + Prompt Design + Generation

Not literally as a formal definition — but as a **practical mental model** for building and debugging.

GENERATION · THE FULL FUNCTION USE THE CHAT TEMPLATE

Generate the Answer

COLAB CELL — END-TO-END GENERATE

```python
def generate_answer(question, k=3):

    retrieved = retrieve(question, k=k)
    context = build_context(retrieved)
    user_prompt = build_prompt(question, context)

    messages = [{"role": "user", "content": user_prompt}]

    text = tokenizer.apply_chat_template(
        messages, tokenize=False,
        add_generation_prompt=True
    )

    inputs = tokenizer(text, return_tensors="pt").to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=150,
            do_sample=False
        )

    new_tokens = outputs[0][inputs["input_ids"].shape[1]:]
    answer = tokenizer.decode(new_tokens, skip_special_tokens=True)

    return answer, retrieved
```

1-3 Retrieve → build context → build prompt — everything from today so far.

4 **apply_chat_template** formats the message for an instruct model.

5 Tokenize and move inputs to the model's device.

6 **max_new_tokens=150**, **do_sample=False** → deterministic, reproducible answers.

7 Decode **only the new tokens** — never echo the prompt.

Returns **`answer, retrieved`** — so the evidence always travels with the response.

THE MOMENT IT RUNS QUESTION + EVIDENCE + ANSWER

Run the Complete RAG System

COLAB CELL — RUN

```
question = "My router Internet light is red.
           What should I do?"

answer, retrieved = generate_answer(question, k=3)

print("QUESTION:")
print(question)

print("\nANSWER:")
print(answer)
```

CONCEPTUALLY EXPECTED

"Verify the WAN cable, restart the modem and router, wait for synchronization, and contact support if the light remains red. **Source: Internet LED Troubleshooting.** *"*

Exact model wording may vary.

DISPLAY RETRIEVED EVIDENCE WITH THE ANSWER

```
print("\nRETRIEVED SOURCES:")

for r in retrieved:
    print("\nSource:", r["source"],
          "\nScore:", round(r["score"], 4),
          "\nText:", r["text"])
```

Now students can see: **Question + Retrieved Evidence + Final Answer** — much more useful than seeing only the answer.

THE KEY EXPERIMENT VERY IMPORTANT — RUN BOTH WAYS

Compare: Without RAG vs With RAG

Ask the base model directly — no knowledge base. It may produce useful generic advice. But: *"Is it answering from our official knowledge base?"* — **No.** A plausible answer is **not equivalent** to a grounded answer.

STANDALONE LLM	RAG
Depends on parametric knowledge	Uses supplied KB
May give generic response	Can give product-specific response
Source may be unavailable	Source can be shown
May invent unsupported steps	Can be instructed to stay grounded
Harder to update knowledge	KB can be updated

📌 **Key point:** the goal of RAG is not merely to make an answer longer or smarter — it is to make the answer **use relevant external evidence**.

CONTROL · ABSTENTION TESTING THE HONEST EXIT

Test an Unknown Question — Then Add a Threshold

OUR KB DOES NOT CONTAIN THIS

```
question = "Does this router support
           satellite internet?"

answer, retrieved = generate_answer(question, k=3)

print(answer)
```

DESIRED RESPONSE

"I do not have enough information in the provided knowledge base."

This tests **abstention**.

But will abstention always work? Not necessarily — the retriever still returns its **closest** chunks even if none is truly relevant:

1. Router setup
2. Internet LED
3. Firmware

Low relevance — but still Top-3. Connects directly to **Lecture 3**.

Production issue: Top-k always returning something does not mean useful evidence exists.

A SIMPLE ENHANCEMENT

```
def retrieve_with_threshold(query, k=3, min_score=0.30):

    results = retrieve(query, k=k)

    filtered = [r for r in results if r["score"] ⩾ min_score]

    return filtered
```

Important: 0.30 is only an illustrative threshold — validate it for your embedding model and dataset. Do not teach one universal score threshold.

EVALUATION · RETRIEVAL FIRST A SMALL EVALUATION SET

Evaluate Retrieval First

FOUR QUESTION → EXPECTED SOURCE PAIRS

```
evaluation_set = [
    {"question": "How do I reset the router?",
     "expected_source": "Factory Reset Guide"},

    {"question": "Where is the default Wi-Fi password?",
     "expected_source": "Wi-Fi Password Guide"},

    {"question": "What should I do when the Internet
     LED is red?",
     "expected_source": "Internet LED Troubleshooting"},

    {"question": "Can I unplug power during firmware
     update?",
     "expected_source": "Firmware Update Guide"}
]
```

TOP-1 ACCURACY

```
correct = 0

for item in evaluation_set:
    results = retrieve(item["question"], k=1)
    predicted = results[0]["source"]
    is_correct = (predicted == item["expected_source"])
    correct += int(is_correct)
    print(item["question"], "→", predicted, "→", is_correct)

accuracy = correct / len(evaluation_set)
print("Top-1 retrieval accuracy:", accuracy)
```

Later in research/production: Precision@k Recall@k MRR

POSSIBILITY A

Wrong evidence retrieved → the LLM **never had a chance**.

POSSIBILITY B

Correct evidence retrieved → the LLM **still answered incorrectly**.

Different failures → **different fixes**. "First test whether the model received the right evidence. Then test what it did with that evidence."

EVALUATION · DEBUGGING FILL IT BY HAND — THAT IS THE POINT

A Simple Evaluation Table — and the Debug Checklist

QUESTION	EXPECTED	RETRIEVED	RET. OK?	ANS. OK?
Reset router?	Reset Guide	Reset Guide	Yes	Yes
Wi-Fi password?	Password Guide	Password Guide	Yes	Yes
Red LED?	Troubleshooting	Troubleshooting	Yes	Yes
Warranty damage?	Warranty	Warranty	Yes	Yes

Two separate columns — this table **separates Retrieval** `from` **Generation**.

WHEN THE ANSWER IS WRONG — 8 CHECKS

- **S1** Is the information actually present in the knowledge base?
- **S2** Was the text extracted correctly?
- **S3** Was chunking appropriate?
- **S4** Are embeddings meaningful?
- **S5** Did FAISS retrieve the correct chunk?
- **S6** Did Top-k include useful evidence?
- **S7** Was the context constructed correctly?
- **S8** Did the LLM follow the evidence?

📌 **Memory anchor:** `Debug RAG from left to right.`

EXTENSION · REAL KNOWLEDGE SYLLABUS 2.3 — PUBLIC KNOWLEDGE BASES

Add New Documents — Then Go Public (PDF)

TOMORROW WE ADD Parental_Control_Guide

New Document → Chunk → Embed → Add to Vector Index

We do **not** need to retrain Qwen — new queries can retrieve it immediately.

The fundamental advantage of RAG: **knowledge can change independently of model weights**.

REPLACE THE CLASSROOM LIST WITH A PUBLIC DOCUMENT

- Equipment manuals
- Software documentation
- Open-source project FAQs
- Public institutional handbooks

For the first lab, keep the document **modest enough to inspect manually**.

IF THE KNOWLEDGE SOURCE IS A PDF

```
!pip -q install pypdf
from pypdf import PdfReader

reader = PdfReader("manual.pdf")
pages = []

for page_no, page in enumerate(reader.pages):
    text = page.extract_text()
    if text:
        pages.append({"text": text,
                      "page": page_no + 1,
                      "source": "manual.pdf"})

# PDF → pages → chunk each page → embedding
```

PRESERVE PAGE METADATA

```
for page in pages:
    pieces = chunk_text(page["text"],
                        chunk_size=120, overlap=20)
    for piece in pieces:
        pdf_chunks.append({"text": piece,
            "source": page["source"],
            "page": page["page"]})

# retrieved evidence can say:
#   Source: manual.pdf · Page: 17
```

Page-level metadata **directly supports citations** — Lecture 4's traceability, now in code.

WARNINGS + PERSPECTIVE

EIGHT ANTIPATTERNS, THEN THE FRAMEWORK VIEW

What Students Should Not Do

M1 Start with LangChain without understanding retrieval.

M2 Never inspect retrieved chunks.

M3 Evaluate only final-answer fluency.

M4 Use huge chunks without testing.

M5 Assume Top-k = 10 must be better than 3.

M6 Treat the similarity score as truth.

M7 Assume citations automatically mean the answer is supported.

M8 Put sensitive private documents into a public service without access controls.

ONLY AFTER THE MANUAL PIPELINE — SHOW

LangChain / LlamaIndex

can simplify:

- Document Loading
- Text Splitting
- Embedding
- Vector Store
- Retrieval
- Prompt Construction
- LLM Invocation

`Load → Chunk → Embed → Store → Retrieve → Augment → Generate`

STILL EXISTS UNDERNEATH

ONE PAGE, WHOLE SYSTEM THE NOTEBOOK ON A SINGLE SLIDE

Full Working Mental Model

KNOWLEDGE BASE

Public Documents / FAQ → LOAD → CHUNK → EMBED → STORE → FAISS INDEX

USER QUESTION

User Question → EMBED → RETRIEVE → Top-k Chunks → Augment → Generator LLM → Answer → Source + Evaluation

SUCCESS CRITERION — EXPLAIN THIS CONCEPTUALLY

```
results = retrieve(question)
context = build_context(results)
prompt  = build_prompt(question, context)
answer  = LLM(prompt)
```

Find Evidence
↓ Prepare Context
↓ Give Evidence + Question to LLM
↓ **Generate Answer**

If students understand **why those four operations exist** — rather than merely running the notebook — the practical has successfully completed **Module 2**.

MODULE 2 COMPLETE FIVE LECTURES, ONE ARC

Complete Module 2 Recap

L1 **Why RAG?**
LLM limitations + external knowledge

L2 **How does RAG work?**
Load · Chunk · Embed · Store · Retrieve · Augment · Generate

L3 **How do we improve RAG?**
Similarity · Chunking · Top-k · Metadata · Reranking · Evaluation

L4 **Where is RAG used?**
Healthcare · Finance · Support · Education

L5 **Can we actually build it?**
Yes.

BLACKBOARD — KEEP ONLY THESE SIX

1. Load → Chunk → Embed → Store
2. Question → Embed → Retrieve
3. Retriever finds. Generator writes.
4. Evidence + Question → LLM
5. Evaluate: Retrieval ≠ Generation
6. Debug RAG left → right

ONE SENTENCE STUDENTS SHOULD REMEMBER

"A practical RAG system converts external documents into searchable embeddings, retrieves the most relevant evidence for each user query, adds that evidence to the LLM prompt, generates a grounded response, and evaluates retrieval and generation separately."

THE FINAL MODULE FORMULA

Useful RAG = Good Knowledge + Good Retrieval + Grounded Generation + Evaluation

"A vector database does not make a RAG system good. An LLM does not make a RAG system good.
The complete pipeline does."